

Political Intelligence & Grievance Management Platform

| Interview Preparation Guide

Full Stack Developer Assessment — Starkin Solutions

Candidate: Delli Babu | Stack: React + Vite + TypeScript, Node.js + Express + TypeORM + PostgreSQL
Contents: Project Walkthrough · Architecture · Database Design · Code Review · 70+ Interview Q&A

Table of Contents

1. 1. One-Minute Project Pitch
2. 2. Architecture & Tech Stack
3. 3. Database Design (14 Entities + ERD)
4. 4. Backend Walkthrough (Auth, RBAC, Complaint Lifecycle, Sockets, Files, Dashboard)
5. 5. Frontend Walkthrough (Routing, Auth Context, Socket Context, Data Fetching)
6. 6. Code Review — Strengths & Improvement Areas
7. 7. Interview Q&A — Project Walkthrough & Behavioral
8. 8. Interview Q&A — Authentication, JWT & RBAC
9. 9. Interview Q&A — Database & TypeORM
10. 10. Interview Q&A — API Design, Validation & Error Handling
11. 11. Interview Q&A — React & Frontend
12. 12. Interview Q&A — Real-Time (Socket.io)
13. 13. Interview Q&A — System Design, Scaling & Bonus Features
14. 14. Tricky / Code-Specific Questions (Things They Might Probe)

1. One-Minute Project Pitch

"I built a production-style **Grievance Management Platform** for Andhra Pradesh citizens to file, track, and resolve civic complaints (potholes, healthcare access, water supply, electricity, education issues). It's a full MERN-style stack but with **PostgreSQL + TypeORM** instead of Mongo, and **TypeScript** across both frontend and backend.

The platform supports **four roles** — Super Admin, State Admin, District Admin, and Volunteer — each with different visibility and permissions enforced both at the API layer (RBAC middleware) and the UI layer (route guards). Citizens can submit complaints anonymously from a public landing page with file attachments (images/PDF/video), and track status later using a generated ticket number like `GRV-2026-10001`, with no login required.

Internally, admins assign complaints to volunteers or district admins, the assignee gets a **real-time Socket.io notification**, and every status change is recorded in an immutable complaint timeline (`complaint_updates`) and a security **audit log** (who did what, when, from which IP). The dashboard uses **Recharts** to visualize status breakdowns, category distribution, district-wise load, and a 6-month trend — all scoped by the logged-in user's role."

Key Differentiators to Mention

- **Security:** short-lived (15 min) JWT access tokens + HTTPOnly, SameSite=strict refresh token cookies (7 days), bcrypt password hashing, axios interceptor auto-refresh-on-401 with request queuing.
- **RBAC done at two layers:** Express middleware (`authorize()`) checks permission arrays from the DB-backed `roles.permissions` jsonb column; React route guards (`ProtectedRoute`) hide/redirect pages.
- **Data integrity:** 14 normalized relational tables, UUID PKs, foreign keys with cascade rules, unique ticket numbers via `@BeforeInsert` hook.
- **Auditability:** every write (login, create/assign/update/delete complaint, register user) is logged to `audit_logs` with actor, action, entity, meta JSON, and IP.
- **Real-time UX:** Socket.io rooms per-user (`user:<id>`) and per-district (`district:<id>`) so notifications and new-complaint alerts are pushed instantly.

2. Architecture & Tech Stack

2.1 Stack Summary

Layer	Technology	Why
Frontend	React 18 + Vite + TypeScript	Fast HMR, modern build, strong typing across UI
Styling	Tailwind CSS	Utility-first, fast iteration, consistent design tokens
Data fetching	TanStack Query v5 + Axios	Caching, retries, background refetch, request/response interceptors for token refresh
Forms & validation	React Hook Form + Zod	Uncontrolled performant forms + schema validation shared mental model with backend Zod schemas
Routing	React Router v6	Nested routes + layout routes for role-guarded sections
Charts	Recharts	Declarative React charts for dashboard analytics
Real-time	Socket.io-client / Socket.io server	Room-based push notifications, fallback to polling transport
Backend	Node.js + Express + TypeScript	Lightweight, explicit middleware pipeline, easy to reason about
ORM	TypeORM 0.3 (DataSource API)	Decorator-based entities, query builder for analytics, <code>synchronize: true</code> for fast dev iteration
Database	PostgreSQL	Relational integrity, jsonb for flexible permission arrays, UUID + ILIKE search
Auth	JWT (jsonwebtoken) + bcrypt	Stateless access tokens, salted password hashes
File uploads	Multer (disk storage)	Multipart parsing, mimetype whitelist, 50MB limit

2.2 High-Level Request Flow

```
Browser (React SPA, :5173)
  | axios (withCredentials: true, Authorization: Bearer <accessToken>)
  ▼
Express App (:5000)
  | cors() → express.json() → cookieParser() → static /uploads
  |
  ├── /api/auth/*           (login, register, refresh, logout, me)
  ├── /api/complaints/*     (CRUD, assign, update, track – RBAC + multer)
  ├── /api/users/*          (super_admin / manage_users)
  ├── /api/dashboard/*      (role-scoped aggregates via QueryBuilder)
  ├── /api/notifications/*  (per-user inbox)
  └── /api/locations/*       (districts → mandals → villages, categories, roles)
  |
  ▼
TypeORM DataSource — PostgreSQL (grievance_db)
  |
  └─ Socket.io server: rooms "user:<id>" and "district:<id>"
      → emits "notification" and "complaint:new" events
```

2.3 Folder Structure

server/src

- └ config/ data-source.ts (TypeORM config), seeder.ts (demo data)
- └ entities/ 14 TypeORM entity classes (User, Role, Complaint, ...)
- └ middlewares/ authenticate.ts, authorize.ts, upload.ts
- └ controllers/ auth, complaint, dashboard, location, notification, user
- └ routes/ thin Express routers wiring middleware → controller
- └ services/ auditLogService, notificationService, socketService
- └ types/express.ts AuthenticatedRequest + JWT payload shape
- └ index.ts app bootstrap, Socket.io, DB init + auto-seed

client/src

- └ api/client.ts axios instance + refresh-token interceptor
- └ context/ AuthContext, SocketContext, ToastContext
- └ components/ Layout (nav/sidebar), ProtectedRoute (RBAC guard)
- └ pages/ Landing, Tracking, Login, Dashboard, Complaints
(List/Detail/Create), Users, Notifications, Profile

3. Database Design (14 Entities)

3.1 Entity Relationship Summary

```
users —belongs_to→ roles
users —belongs_to→ districts (nullable: super/state admins have none)
users —has_many→ refresh_tokens, audit_logs, notifications

complaints —belongs_to→ categories, districts, mandals, villages(nullable)
complaints —belongs_to→ users (assigned_to, created_by — both nullable)
complaints —has_many→ assignments, complaint_updates, attachments

districts —has_many→ mandals —has_many→ villages (cascading geo hierarchy)
```

3.2 Entity-by-Entity Notes

Entity	Purpose	Notable design points
Role	4 roles, stores <code>permissions: jsonb</code> array	Permission checks are just array membership — no separate join table needed for role↔permission
Permission	Catalog of permission names	Mostly descriptive/reference; actual enforcement reads <code>role.permissions</code>
User	Auth + profile	UUID PK, unique email, bcrypt hash, nullable district (HQ-level roles), <code>is_active</code> soft-disable
District / Mandal / Village	3-level AP geo hierarchy	Mandal FK→District (CASCADE delete), Village FK→Mandal (CASCADE) — deleting a district cleans the tree
Category	Complaint type (Infrastructure, Healthcare...)	<code>is_active</code> flag lets admins retire categories without breaking history
Complaint	Core entity	<code>ticket_number</code> auto-generated via <code>@BeforeInsert</code> (<code>GRV-YYYY-XXXX</code>), <code>status</code> / <code>priority</code> string enums, several eager relations (category, district, mandal, village, assigned_to, created_by)
Assignment	Append-only assignment history	assigned_by / assigned_to / notes / timestamp — kept even if complaint reassigned later
ComplaintUpdate	Timeline / audit trail visible to citizens	status snapshot + comment + actor, ordered ASC for timeline UI
Attachment	Uploaded files	file_type enum (image/pdf/video) derived from mimetype, stores relative <code>/uploads/...</code> URL
Notification	Per-user inbox	<code>is_read</code> boolean, <code>reference_id</code> links back to complaint, paired with Socket.io push
AuditLog	Security/compliance trail	actor (nullable for anonymous), action, entity, entity_id, <code>meta: jsonb</code> , ip_address
RefreshToken	Server-side session control	Stored so logout/refresh can revoke (<code>is_revoked</code>) — JWT alone can't be revoked, this DB row makes it possible

3.3 Why PostgreSQL + TypeORM here?

- The domain is inherently **relational** — complaints reference categories, geography, users, with strict FK integrity (no orphaned complaints).
- `jsonb` on `roles.permissions` and `audit_logs.meta` gives schema flexibility for evolving permission sets / audit payloads without migrations.
- TypeORM's **QueryBuilder** is used for all dashboard aggregates (GROUP BY status/category/district, TO_CHAR for monthly trends) — raw SQL power with parameter binding (SQL-injection safe).
- `synchronize: true` auto-creates/updates tables from entity decorators — fast for assessment/dev; in production this would be replaced by versioned migrations.

4. Backend Walkthrough

4.1 Authentication Flow

1. **Login** (`POST /api/auth/login`): looks up user by email with `role` + `district` relations, `bcrypt.compare()` the password, then:
 - Signs a 15-min **access token** (`signAccessToken`) containing `{id, email, role, permissions, districtId}` .
 - Signs a 7-day **refresh token** (different secret), persists it in `refresh_tokens` table, and sets it as an `HttpOnly, SameSite=strict` cookie.
 - Logs a `LOGIN` audit entry.
2. **Authenticated requests**: client sends `Authorization: Bearer <accessToken>` . `authenticate` middleware verifies signature/expiry with `JWT_SECRET` and attaches `req.user` .
3. **Refresh** (`POST /api/auth/refresh`): reads the `HttpOnly` cookie, verifies it against `JWT_REFRESH_SECRET` , looks the raw token up in `refresh_tokens` (checks `is_revoked` and `expires_at`), and issues a new access token.
4. **Logout**: marks the matching `refresh_tokens` row `is_revoked = true` and clears the cookie — this is what makes JWT logout actually effective server-side.

4.2 RBAC Enforcement

Two middlewares compose per route:

```
// authenticate.ts – verifies JWT, sets req.user
// authorize(["assign_complaint"]) – checks permission array

router.post("/:id/assign", authenticate, authorize(["assign_complaint"]), assignComplaint);
router.delete("/:id", authenticate, authorize(["delete_complaint"]), deleteComplaint);
```

`authorize()` gives `super_admin` an automatic bypass, otherwise checks `requiredPermissions.every(p => req.user.permissions.includes(p))` — permissions come straight from the JWT payload (sourced from `role.permissions` jsonb at login time).

Row-level scoping (beyond yes/no permission checks) is done manually inside controllers, e.g. `listComplaints` :

```
if (role === "district_admin") {
  qb.andWhere("complaint.district_id = :uDistrictId", { uDistrictId: districtId });
} else if (role === "volunteer") {
  qb.andWhere("complaint.assigned_to = :uUserId", { uUserId: userId });
}
```

4.3 Complaint Lifecycle

Step	Endpoint	What happens
1. Citizen files complaint	<code>POST /api/complaints</code> (public, <code>upload.array("attachments", 5) + optionalAuthenticate</code>)	Zod-validates payload, resolves category/district/mandal/village FKs, auto-generates ticket number, saves attachments, writes first <code>complaint_updates</code> row ("registered"), emits <code>complaint:new</code> to <code>district:<id></code> room, notifies district admins, writes audit log

2. Admin assigns	<code>POST /api/complaints/:id/assign</code> (needs <code>assign_complaint</code>)	Validates assignee is a volunteer/district_admin in the <i>same district</i> , creates an <code>Assignment</code> row, sets <code>status = "assigned"</code> if it was pending, writes timeline update, creates a real-time notification for the assignee
3. Volunteer/admin updates	<code>POST /api/complaints/:id/update</code>	Adds a comment; volunteers restricted to setting status only to <code>in_progress</code> / <code>resolved</code> ; sets <code>resolved_at</code> when resolved
4. Generic edit	<code>PUT /api/complaints/:id</code>	Admins can change status/priority; volunteers can change status only (not priority); writes a diff-style comment to the timeline
5. Public tracking	<code>GET /api/complaints/track/:ticket</code> (no auth)	Returns a sanitized view (no citizen phone/email, no internal user refs) plus the public timeline

4.4 File Uploads (Multer)

- Disk storage under `UPLOAD_DIR` (`./uploads`), filename = `fieldname-timestamp-random.ext` to avoid collisions.
- `fileFilter` whitelists mimetypes: jpeg/png/gif/webp, pdf, mp4/mov/mkv/mpeg — rejects everything else with an `Error` (caught by Express error handler).
- 50MB per-file limit; up to 5 files per complaint (`upload.array("attachments", 5)`).
- `Attachment.file_type` derived server-side from `file.mimetype` prefix — never trusts client-provided type.
- Served statically: `app.use("/uploads", express.static(path.resolve(uploadDir)))` .

4.5 Real-Time Notifications (Socket.io)

```
// On connect, client passes userId & districtId as query params
socket.join(`user:${userId}`);
socket.join(`district:${districtId}`);

// Server pushes:
sendNotificationToUser(userId, {...}) → io.to(`user:${userId}`).emit("notification", ...)
sendNewComplaintToDistrict(districtId, {...}) → io.to(`district:${districtId}`).emit("complaint:new", ...)
```

Every `createNotification()` call both persists a `Notification` row (so it survives reload / shows in the inbox) *and* emits a live socket event (so it shows as a toast immediately).

4.6 Dashboard Analytics

All four chart endpoints (`/stats` , `/by-district` , `/by-category` , `/by-status` , `/trend`) use TypeORM **QueryBuilder** with `GROUP BY` + `COUNT()` , scoped for `district_admin` via a shared `applyDashboardScope()` helper. The trend endpoint uses `TO_CHAR(complaint.created_at, 'YYYY-MM')` and then back-fills the last 6 months client-side so months with zero complaints still appear on the chart (no gaps in the line graph).

4.7 Auto-Seeding

On boot, `index.ts` checks `roles` table count; if 0, runs `runSeeder()` which truncates everything (dependency-ordered) and inserts: 9 permissions, 4 roles, 5 categories, 5 districts × 3 mandals × 3 villages, 4 demo users (super_admin / state_admin / district_admin / volunteer, all `Admin@123`), and 10 realistic sample complaints with timeline history.

5. Frontend Walkthrough

5.1 App Composition & Routing

`App.tsx` nests providers: `QueryClientProvider` → `BrowserRouter` → `AuthProvider` → `SocketProvider` → `ToastProvider`, then `Routes`:

- **Public:** `/` (landing + complaint form), `/track`, `/login`.
- **Protected shell:** everything else wrapped in `<ProtectedRoute>` + `<Layout>` (nav/sidebar).
- **Permission-gated nested route:** `/dashboard` requires `view_dashboard`, `/users` requires `manage_users` — both via a second `<ProtectedRoute requiredPermissions={[...]>` wrapping just that route.
- **Fallback:** unknown authenticated paths redirect to `/complaints`.

5.2 AuthContext

- On mount, calls `POST /auth/refresh` (using the `HttpOnly` cookie) to silently obtain a fresh access token, then `GET /auth/me` to hydrate the user profile — this is what gives "stay logged in after page refresh" behavior *without storing the access token in localStorage*.
- `login()` stores the access token in a module-level variable (`setLocalAccessToken`) — deliberately **not** `localStorage`, reducing XSS token-theft risk.
- `hasPermission(perm)` — `super_admin` always true, else checks `user.permissions.includes(perm)`, used by `ProtectedRoute` and conditionally rendered UI (e.g., "Delete" button only for users with `delete_complaint`).
- Listens for a custom `"auth:logout"` window event dispatched by the axios interceptor when a silent refresh fails — keeps context state in sync with interceptor-driven logouts.

5.3 Axios Client & Token Refresh Interceptor

- Request interceptor attaches `Authorization: Bearer <accessToken>` from the in-memory token.
- Response interceptor: on `401` (and not already retried, and not the login/refresh endpoints themselves), it:
 1. Sets an `isRefreshing` flag and calls `/auth/refresh`.
 2. On success, updates the in-memory token and **replays all queued requests** that piled up while refreshing (via a `refreshSubscribers` array) — prevents a "thundering herd" of parallel refresh calls.
 3. On failure, clears the token and dispatches `auth:logout`.

5.4 SocketContext

Opens a socket connection only when `user` is present, passing `userId` and `districtId` as handshake query params so the server can join the right rooms. Falls back from `websocket` to `polling` transport. Cleans up (disconnect) on user logout or unmount.

5.5 Pages

Page	Highlights
LandingPage	Public complaint form — React Hook Form + Zod, cascading District→Mandal→Village selects fetched via TanStack Query, file input for attachments (multipart)
TrackingPage	Ticket-number lookup, renders sanitized complaint + timeline, no auth

DashboardPage	Recharts: pie (status), bar (category), horizontal bar (district), line (6-month trend) — all role-scoped by backend
ComplaintsListPage	Filters: status, priority, district (admin-only), category, search (ILIKE on ticket/title/citizen)
ComplaintDetailPage	Timeline, attachments, assignment selector (role-gated), status/priority update forms
UserManagementPage	super_admin only — CRUD users, assign role + district
NotificationsPage	Inbox fed by REST + live Socket.io <code>"notification"</code> events

6. Code Review — Strengths & Improvement Areas

Be ready to discuss these honestly — interviewers respect candidates who can critique their own code.

6.1 Strengths

Consistent response shape — every endpoint returns `{ success, ...payload }` or `{ success: false, message }`, making frontend error handling predictable.

Defense in depth for auth — short-lived access tokens, revocable refresh tokens stored server-side, HttpOnly cookies, bcrypt with per-user salt, in-memory (not localStorage) access token.

RBAC at multiple layers — route-level permission middleware + row-level query scoping + frontend route guards + conditional rendering.

Full audit trail — every mutating action logs actor/action/entity/IP, satisfying the "Audit Logs" requirement properly (not just a stub).

Type safety end-to-end — shared-shape DTOs (e.g., `UserSessionPayload`), Zod validation on the create-complaint payload mirrors the entity constraints.

6.2 Honest Improvement Areas (good to pre-empt)

TypeORM `synchronize: true` in `data-source.ts` is convenient for dev/demo but unsafe for production — schema drift should be managed via generated migrations (`typeorm migration:generate`).

Eager relations on `Complaint` (category, district, mandal, village, assigned_to, created_by all `eager: true`) combined with explicit `leftJoinAndSelect` in `listComplaints` works but can over-fetch on high-traffic list endpoints — could switch to lazy + selective joins for performance at scale.

No pagination on `GET /api/complaints` — `getMany()` returns the full result set; for a state with thousands of complaints this needs `skip/take` + total count.

Refresh tokens accumulate — no scheduled cleanup job for expired/revoked rows in `refresh_tokens` (would grow unbounded over time; a cron/background job would prune them).

Rate limiting / brute-force protection absent on `/api/auth/login` — would add `express-rate-limit` in production.

Bonus items not yet implemented: Docker, Swagger/OpenAPI docs, email notifications, CI/CD, PDF/Excel export, background job queue (e.g., BullMQ) — all explicitly called out as optional/bonus in the brief, good to mention as "next steps" roadmap.

Refresh-token rotation — current refresh tokens are long-lived (7 days) and not rotated on use; rotating on each refresh (and revoking the old one) would reduce replay-attack windows.

6.3 If Asked "What Would You Do With More Time?"

- Add pagination + server-side sorting to complaint listings.
- Swap `synchronize: true` for migrations before any production deploy.
- Add Swagger/OpenAPI spec (the bonus item) — Express routes are already cleanly separated, easy to annotate.
- Add a background job (node-cron or BullMQ) for refresh-token cleanup and "complaint SLA breach" email reminders.

- Add integration tests (Jest + Supertest) for the auth and RBAC flows — highest-risk surface area.
- Containerize with Docker Compose (postgres + server + client) for one-command setup.

7. Interview Q&A — Project Walkthrough & Behavioral

Q: Walk me through what happens when a citizen submits a complaint.

A: They fill the public landing-page form (title, description, category, district→mandal→village cascading selects, contact info, optional attachments). React Hook Form + Zod validate client-side. On submit, axios sends a `multipart/form-data` POST to `/api/complaints` (no auth required). The server re-validates with Zod, resolves all FK relations, auto-generates a `GRV-YYYY-XXXX` ticket number via a `@BeforeInsert` hook, saves the complaint + any attachments, writes the first timeline entry, emits a `complaint:new` Socket.io event to that district's room, creates persistent notifications for district admins, writes an audit log, and returns the ticket number so the citizen can track it later.

Q: Why did you choose PostgreSQL + TypeORM over MongoDB/Mongoose, given many MERN tutorials use Mongo?

A: The assessment required it, but it's also the right fit: the domain is highly relational (complaints reference categories, a 3-level geography hierarchy, and users with different roles), and we need strong referential integrity (e.g., you can't assign a complaint to a non-existent user) plus aggregate reporting (GROUP BY for dashboards) — both areas where Postgres + SQL excel over a document store.

Q: What was the hardest part of this project?

A: Getting the RBAC model right across three layers consistently — DB-stored permissions (jsonb), Express middleware enforcement, and React route/UI guards — while also adding row-level scoping (e.g., a `district_admin` can only see their district's complaints, a volunteer only their assigned ones) on top of the coarse-grained permission checks.

Q: How would you explain this project to a non-technical interviewer?

A: "It's a digital complaint box for citizens — like filing a pothole report and getting a receipt number to check progress later, while government staff at state/district level see a dashboard of all complaints, assign them to field volunteers, and everything is logged for accountability."

Q: If this had to support all of Andhra Pradesh (millions of complaints), what's the first thing you'd change?

A: Add pagination + indexes on frequently filtered columns (`status`, `district_id`, `category_id`, `created_at`), move from `synchronize: true` to migrations, add caching (Redis) for dashboard aggregates which don't need to be real-time-accurate, and consider read replicas for the analytics queries so they don't compete with write traffic from complaint submissions.

Q: How did you test that everything actually works end-to-end?

A: I ran both dev servers, verified TypeORM connected and synced the schema against a local Postgres instance, ran the seeder to populate roles/users/sample complaints, then hit the API directly with curl: logged in as the seeded super admin, fetched dashboard stats and complaint lists with the JWT, and submitted a brand-new public complaint via multipart form data to confirm the ticket-number generation and tracking endpoint worked end-to-end.

8. Interview Q&A — Authentication, JWT & RBAC

Q: Why two tokens (access + refresh) instead of one long-lived JWT?

A: A single long-lived JWT can't be revoked — if stolen, it's valid until expiry. Splitting into a short-lived (15 min) access token (sent in headers, used for API calls) and a long-lived (7 day) refresh token (HttpOnly cookie, used only to mint new access tokens) limits the blast radius of a stolen access token, and the refresh token's server-side record (`refresh_tokens` table with `is_revoked`) lets us actually invalidate sessions on logout.

Q: Where do you store the access token on the client, and why?

A: In a module-level JS variable inside `api/client.ts` (in memory), not localStorage/sessionStorage. This avoids exposing it to XSS-based token theft (any injected script reading localStorage). The tradeoff is it's lost on full page reload, which is why `AuthContext` calls `/auth/refresh` on mount to silently re-obtain it using the HttpOnly cookie.

Q: Why is the refresh token in a cookie but the access token in a header?

A: The refresh token needs to be inaccessible to JavaScript (HttpOnly) since it's the longer-lived, higher-value credential — cookies are the only browser storage that supports HttpOnly. The access token is short-lived and needs to be attached to cross-origin API calls explicitly, which headers do cleanly without CSRF concerns (cookies sent automatically would need CSRF protection).

Q: How does logout actually invalidate a session, given JWTs are stateless?

A: Logout looks up the refresh token row in the DB by its value and sets `is_revoked = true`, then clears the cookie. Any future `/auth/refresh` call with that token will be rejected (`tokenRecord.is_revoked` check), so even though the still-valid 15-minute access token could theoretically be replayed until it expires, no new access tokens can be minted — capping exposure to at most 15 minutes.

Q: Explain your RBAC model — roles vs permissions.

A: 4 roles (`super_admin`, `state_admin`, `district_admin`, `volunteer`) each map to a `permissions` jsonb array (e.g., `create_complaint`, `assign_complaint`, `manage_users`, `view_dashboard`, `delete_complaint`). At login, the user's role's permissions are embedded into the JWT payload. The `authorize(requiredPermissions)` middleware checks `permissions.every(p => req.user.permissions.includes(p))`, with `super_admin` as an automatic bypass. This is coarse-grained "can you do X at all" — row-level "which records can you see" is handled separately per-controller using `role + districtId / userId` from the JWT.

Q: A district_admin tries to view a complaint from another district via `GET /api/complaints/:id` — what happens?

A: `getComplaintById` fetches the complaint, then checks `if (role === "district_admin" && complaint.district.id !== districtId) return 403 Forbidden`. Same pattern for volunteers checking `complaint.assigned_to.id !== userId`. So even with a valid token and a guessable UUID, cross-district/cross-user access is blocked at the controller level — not just hidden in the list view.

Q: How are passwords stored and verified?

A: `bcrypt.genSalt(10)` + `bcrypt.hash(password, salt)` on register/seed; login uses `bcrypt.compare(password, user.password)`. Bcrypt's per-hash salt and adjustable cost factor (10 rounds here) make offline brute-forcing of the hash table expensive even if the DB is compromised.

Q: What happens if the access token expires mid-session while the user is active?

A: The next API call returns 401 "Access token has expired or is invalid". The axios response interceptor catches this (excluding the login/refresh endpoints themselves to avoid loops), calls `/auth/refresh` using the HttpOnly cookie, gets a new access token, updates the in-memory token, and **transparently retries the original failed request** — the user never sees an error. Concurrent requests during the refresh are queued and replayed once the new token arrives.

Q: Is this implementation vulnerable to CSRF?

A: The refresh cookie is set with `sameSite: "strict"`, which prevents the browser from sending it on cross-site requests — that's the primary CSRF mitigation here. The access token is sent via an explicit `Authorization` header (not a cookie), so it's immune to CSRF by design (CSRF exploits ambient cookie-based auth). In production over HTTPS, `secure: true` would also be set (currently gated on `NODE_ENV === "production"`).

9. Interview Q&A — Database & TypeORM

Q: Why UUIDs instead of auto-increment integers for primary keys?

A: UUIDs avoid leaking sequential information (e.g., guessing how many complaints exist or enumerating ticket IDs), are safe to generate client-side or across distributed services without collision, and work well as opaque public identifiers in URLs (`/complaints/:id`) without exposing business volume.

Q: How do you ensure each complaint gets a unique, human-friendly ticket number?

A: A `@BeforeInsert()` hook on the `Complaint` entity generates `GRV-{year}-{5-digit-random}` if not already set, and the column has a `unique` constraint at the DB level as a backstop. For the seeder, deterministic incrementing numbers are used instead to guarantee no collisions across the 10 seed rows.

Q: What's the difference between `find()` / `findOne()` and `createQueryBuilder()` in this codebase — when did you use which?

A: `findOne` / `find` with a `relations` array are used for simple lookups by ID/email where eager-loading specific relations is enough (e.g., `getComplaintById`, `login`). `createQueryBuilder` is used wherever I need **conditional WHERE clauses built dynamically** (role-based filters, search, multiple optional query params in `listComplaints`) or **aggregates** (`GROUP BY` / `COUNT` in the dashboard controller) — things the repository API can't express cleanly.

Q: How do you prevent SQL injection in the search filter (`ILIKE '%...%'`)?

A: TypeORM's QueryBuilder uses **parameterized queries** — `andWhere("complaint.title ILIKE :search", { search: `\\%${search}%` })` passes the value as a bound parameter, not string-concatenated SQL, so the driver escapes it safely regardless of special characters in the search term.

Q: Explain the cascade behavior in your geography hierarchy.

A: `Mandal.district` and `Village.mandal` are `ManyToOne` with `onDelete: CASCADE` at the FK level, so deleting a District cascades to delete its Mandals and their Villages. Complaint's FKs to district/mandal/village use `NO ACTION` (default) — you wouldn't want deleting a village to silently delete historical complaint records, so that's intentionally NOT cascaded.

Q: Why store `permissions` as jsonb on Role instead of a many-to-many `role_permissions` join table?

A: Permission sets per role are small, static-ish lists checked as a whole at runtime (array membership), not queried relationally (e.g., "find all roles with permission X" isn't a feature). jsonb gives flexibility to add/remove permissions per role without schema migrations, and avoids an extra join on every authenticated request. If permission management became a dynamic admin feature with cross-cutting queries, I'd normalize it into a join table.

Q: What does `synchronize: true` do and why is it risky in production?

A: On every app start, TypeORM diffs the entity definitions against the actual DB schema and auto-applies `CREATE TABLE` / `ALTER TABLE` to match — great for rapid iteration (no manual migration writing during development), but in production it can run destructive/unexpected DDL (e.g., dropping a column you renamed) without review. The fix is `synchronize: false` + TypeORM migrations (`migration:generate` / `migration:run`) reviewed and applied deliberately.

Q: How are attachments linked to complaints, and how do you know if a file is an image, PDF, or video?

A: Each uploaded file becomes an `Attachment` row with a `complaint` FK, `file_url` (e.g., `/uploads/attachments-167...-3.png`), `file_name`, `file_size`, and `file_type`. The type is derived server-side from `file.mimetype` (image/*, video/*, else pdf) at upload time — never trusting a client-supplied type field, which avoids spoofing.

10. Interview Q&A — API Design, Validation & Error Handling

Q: How is input validated, and why Zod specifically?

A: Zod schemas (e.g., `complaintSchema` in `complaintController.ts`) define type-safe runtime validation: min-length strings, regex for Indian 10-digit mobile numbers starting 6-9, UUID format checks for FK ids, enum checks for priority. `safeParse()` returns a discriminated result so I can return `400` with `parsed.error.format()` (field-level error map) without throwing. Zod is also used on the frontend with React Hook Form's `zodResolver`, so the same validation "shape" mentally maps between client and server (DRY-ish, even though schemas are currently duplicated rather than shared via a monorepo package).

Q: What's your global error-handling strategy?

A: Every controller wraps its logic in try/catch and returns `{ success: false, message: error.message }` with an appropriate status code (400 validation, 401 auth, 403 forbidden, 404 not found, 500 unexpected). There's also an Express-level global error middleware (last in the chain in `index.ts`) that catches anything that slips through (e.g., Multer file-filter errors thrown synchronously), logs it server-side, and in development includes the stack trace in the response.

Q: How does the create-complaint endpoint support both logged-in admins and anonymous citizens?

A: `optionalAuthenticate` middleware (vs. the strict `authenticate`) tries to verify the Bearer token if present but doesn't reject the request if it's missing/invalid — it just leaves `req.user` undefined. The controller then branches: if `req.user` exists, `created_by` is set to that user (admin filing on someone's behalf); otherwise `created_by` is null and the initial timeline comment is attributed to the system/super-admin account.

Q: How would you version this API if you needed breaking changes later?

A: Prefix routes with `/api/v2/...` mounted alongside `/api/v1` (current `/api`), or use a header-based version negotiation. Given the current flat `/api/*` structure with thin route files per resource, adding a versioned mount point is a low-effort change — each route file is already isolated.

Q: Why return the full `permissions` array to the client at login instead of just the role name?

A: It lets the frontend do instant, network-call-free permission checks (`hasPermission()` in `AuthContext`) for conditional rendering — e.g., hiding the "Delete" button for non-super-admins — without re-deriving role→permission mappings on the client (which would duplicate business logic) or making a round trip per UI decision.

11. Interview Q&A — React & Frontend

Q: Why TanStack Query instead of plain `useEffect` + `useState` for data fetching?

A: TanStack Query gives caching (so switching between Complaints List and Detail doesn't always refetch), automatic background refetching, request deduplication, built-in loading/error states, and easy cache invalidation after mutations (e.g., after assigning a complaint, invalidate the complaints list query so it refreshes). It removes a lot of boilerplate and race-condition bugs that hand-rolled `useEffect` fetching tends to have.

Q: How does `ProtectedRoute` work, and how does it differ from backend authorization?

A: `ProtectedRoute` is a layout route component: if `loading` (auth state still resolving), it shows a spinner; if no `user`, it `<Navigate to="/login">`; if `requiredPermissions` are passed and `hasPermission()` fails, it redirects (e.g., to `/complaints`). This is purely a **UX guard** — it prevents flicker/unwanted navigation but is not a security boundary; the real enforcement is server-side `authorize()` middleware, since a user could otherwise hit the API directly.

Q: Why React Hook Form over Formik or controlled state?

A: RHF is largely uncontrolled (refs-based), so large forms like the complaint submission form (10+ fields, file inputs, cascading selects) don't re-render on every keystroke — better performance. Combined with `zodResolver`, validation errors are derived automatically from the same Zod schema used to type the form values.

Q: How do the cascading District → Mandal → Village dropdowns work?

A: Districts are fetched once on mount via `/api/locations/districts`. When a district is selected, a TanStack Query for `/api/locations/districts/:id/mandals` is enabled (using the `enabled` option keyed on the selected district id), and similarly mandal selection enables the villages query. Changing the district resets the mandal/village form fields to avoid stale invalid combinations.

Q: How does the Toast/notification UI relate to Socket.io?

A: `ToastContext` exposes a `showToast()` function. A component subscribed via `useSocket()` listens for the `"notification"` event and calls `showToast()` when one arrives, while `NotificationsPage` also fetches the persisted list from `/api/notifications` via TanStack Query — so the same event drives both an ephemeral toast and a durable inbox entry.

Q: What would happen if you removed `withCredentials: true` from the axios instance?

A: The browser would stop sending the HttpOnly refresh-token cookie to the server on `/auth/refresh` and `/auth/login` (cookie-setting) requests, breaking silent re-authentication on page reload and effectively forcing users to log in every time they refresh the page.

Q: How is Tailwind configured here, and why utility-first CSS for this project?

A: `tailwind.config.js` + `postcss.config.js` with the standard content globs over `src/**/*.{ts,tsx}` for tree-shaking unused classes. For a multi-page admin-style app with many similar card/table/badge layouts (status badges, priority pills), utility classes let me iterate on layout quickly and keep styling co-located with markup, without maintaining a growing separate CSS file per page.

12. Interview Q&A — Real-Time (Socket.io)

Q: Why Socket.io rooms instead of broadcasting every event to all connected clients?

A: Rooms (`user:<id>` , `district:<id>`) let the server target exactly the clients who need an event — a volunteer's assignment notification shouldn't be broadcast to every connected admin in the state. It also reduces bandwidth and avoids leaking information across districts/roles via the socket channel (which would otherwise bypass the REST-layer RBAC entirely).

Q: How does the client join the correct rooms?

A: When `SocketContext` opens a connection (only once `user` is loaded from `AuthContext`), it passes `userId` and `districtId` as Socket.io handshake `query` params. The server's connection handler reads `socket.handshake.query` and calls `socket.join(`user:${userId}`)` and, if present, `socket.join(`district:${districtId}`)` .

Q: Is there any authentication on the socket connection itself?

A: Currently the room-join trusts the `userId` / `districtId` query params as sent by the client — this is a known simplification. A hardening step would be to pass the JWT access token in the socket handshake (`auth` option) and verify it server-side before joining rooms, so a malicious client can't simply pass an arbitrary `userId` to snoop on another user's notifications. Good to proactively mention this as a "if I had more time" security improvement.

Q: What happens to a notification if the recipient is offline when it's created?

A: `createNotification()` always persists a `Notification` row first (durable), then attempts the Socket.io emit. If nobody is in that `user:<id>` room, the emit is a no-op — no error, no retry needed — and the user simply sees it next time they open `/notifications` (or reconnect and the inbox is fetched via REST).

Q: Two events are emitted: `"notification"` and `"complaint:new"` . What's the difference?

A: `"notification"` is a personal, persisted inbox item for one specific user (e.g., "you've been assigned ticket X"). `"complaint:new"` is an ephemeral district-wide broadcast (no DB row) used to let district dashboards/lists live-update when any new complaint lands in their district — more like a "refresh your view" signal than a personal message.

13. Interview Q&A — System Design, Scaling & Bonus Features

Q: The brief lists Docker, Swagger, Email, CI/CD, and Background Jobs as bonus. If asked to add one quickly, which would you pick and why?

A: Swagger/OpenAPI — the route/controller separation is already clean (one file per resource), so annotating with `swagger-jsdoc` comments or generating from Zod schemas (`zod-to-openapi`) is mostly additive, low-risk, and directly satisfies the "API Documentation" deliverable requirement without touching business logic.

Q: How would you add email notifications without blocking the request/response cycle?

A: Mirror the existing `createNotification()` pattern but push to a queue (BullMQ/Redis or even a simple in-process async fire-and-forget with retry) instead of calling an SMTP API synchronously inside the controller — so a slow/failed email provider never delays the complaint-assignment response to the admin.

Q: How would you Dockerize this?

A: `docker-compose.yml` with three services: `postgres` (official image, volume for data persistence, seeded via the existing `npm run seed` as an init step), `server` (multi-stage build: `npm ci` → `tsc` → run `dist/index.js`), and `client` (Vite build → served via nginx or a static file server). Environment variables map directly from the existing `.env.example` files, so no code changes needed — just configuration.

Q: How would PDF/Excel export of complaints work?

A: A new endpoint (e.g., `GET /api/complaints/export?format=pdf|xlsx`) reusing the same role-scoped query logic as `listComplaints`, then piping results through `exceljs` (Excel) or `pdfkit` / `puppeteer` (PDF) to stream a generated file as the response with appropriate `Content-Disposition` headers — ideally offloaded to a background job for large exports so the HTTP request doesn't time out.

Q: How would you add CI/CD for this repo?

A: GitHub Actions workflow on push/PR: install deps for both `client` and `server`, run `tsc --noEmit` for both (type-check gate — already passes cleanly), run any test suites, then on merge to `main`, build artifacts and deploy (e.g., server to a container host, client static build to a CDN/S3+CloudFront or Vercel/Netlify).

Q: Where are the biggest performance risks as data grows?

A: 1) `listComplaints` with no pagination — `getMany()` on tens of thousands of rows. 2) Eager relations on `Complaint` causing N+1-ish joins on every fetch even when not needed. 3) Dashboard aggregate queries running against the live transactional table — at scale these should hit a read replica or a materialized/aggregated summary table refreshed periodically. 4) No DB indexes explicitly defined beyond PK/FK/unique — would add composite indexes on `(district_id, status)` and `(category_id, status)` for the most common filter combinations.

Q: How would you handle a surge of complaints during, say, a natural disaster?

A: The public submission endpoint is the highest-traffic, lowest-trust surface — I'd add rate limiting per IP, possibly a CAPTCHA, and ensure file uploads are streamed to object storage (S3) rather than local disk so the app server stays stateless and horizontally scalable. The Socket.io layer would need a shared adapter (Redis adapter) if running multiple server instances, so room broadcasts reach users connected to any instance.

14. Tricky / Code-Specific Questions (Things They Might Probe)

Q: In `assignComplaint`, why must the assignee's district match the complaint's district — and what if a `state_admin` needs to assign across districts?

A: The check `if (assignee.district && assignee.district.id !== complaint.district.id) return 400` enforces that volunteers/district admins only get work from their own jurisdiction — prevents accidental cross-district misrouting. `state_admin` / `super_admin` users have `district: null`, so that condition is skipped for them entirely (the `assignee.district &&` guard), meaning HQ-level roles could theoretically be assigned anything — though in practice assignees are filtered to volunteers/district_admins, who do have districts.

Q: In `updateComplaint`, why does setting status to anything other than "resolved" explicitly null out `resolved_at`?

A: To handle the case where a ticket is marked resolved and then re-opened (status changed back to `in_progress`) — `resolved_at` should reflect the *current* resolution state, not a stale historical timestamp from a previous resolution, which would otherwise make "time to resolve" analytics incorrect.

Q: Why does `addUpdate` restrict volunteers to only `in_progress` or `resolved` status transitions?

A: Volunteers are field workers — they shouldn't be able to `reject` a complaint or set it back to `pending` / `assigned` (those are administrative decisions). This enforces a one-directional workflow for the lowest-privilege role while still letting them report progress and completion.

Q: The seeder truncates tables in a specific order — why does order matter, and what would break if reversed?

A: Tables are truncated child-first (complaint_updates, assignments, attachments, complaints... down to permissions/categories/districts last) because of FK constraints — even with `CASCADE` on the TRUNCATE statement, truncating a parent table referenced by a not-yet-truncated child with a restrictive FK would error. Truncating leaf tables first avoids relying on cascade behavior at all.

Q: What would happen if two requests tried to register the same email simultaneously?

A: The application-level check (`userRepo.findOne({where:{email}})` returning existing user → 400) has a race condition window between the check and the insert. The actual safety net is the `UNIQUE` constraint on `users.email` at the DB level — the losing request's `INSERT` would throw a unique-violation error, caught by the try/catch and returned as a 500 (could be improved to detect the specific Postgres error code and return a friendlier 409 Conflict).

Q: Why is `citizen_email` optional but `citizen_phone` required and regex-validated?

A: Phone is the primary contact/identity channel for most citizens filing civic complaints (more universal than email in this context) and is validated against the Indian mobile number format (`^[6-9]\d{9}$` — starts 6-9, 10 digits total). Email is a nice-to-have for additional updates, hence `.optional().or(z.literal(""))` to accept an empty string from the form without failing validation.

Q: The dashboard's `getTrend` endpoint manually back-fills 6 months of data — why not just return whatever the GROUP BY gives?

A: A raw `GROUP BY TO_CHAR(created_at, 'YYYY-MM')` only returns months that actually *have* complaints — if March had zero complaints, it's simply absent from the result set, which would make a Recharts line graph skip/connect across the gap misleadingly. The code builds a full 6-month `monthNames` scaffold and fills in `0` for any month missing from `trendMap`, guaranteeing a continuous x-axis.

Q: Why does `createComplaint` attribute the initial timeline comment to "admin@grievance.com" when the submitter is anonymous?

A: `ComplaintUpdate.updated_by` is a required (non-nullable) relation to `User` — every timeline entry needs an actor for audit consistency. Since anonymous citizens aren't `User` records, the system falls back to the seeded super-admin account as a "system" actor for that first auto-generated "Complaint successfully registered" entry. An alternative would be making `updated_by` nullable with a separate "system-generated" flag, but that would require an entity/migration change.

Q: What's the significance of `optional().or(z.literal(""))` appearing on `village_id` and `citizen_email` ?

A: HTML forms submit empty `<select>` / `<input>` values as empty strings `""`, not `undefined`. Zod's `.optional()` alone only accepts `undefined` — without `.or(z.literal(""))`, an unselected village or blank email field would fail the UUID/email format validation. This pattern bridges "HTML form empty value" with "Zod optional field" semantics.

Q: If I gave you a JWT and asked you to explain exactly what's inside it for this app, what would you say?

A: Header (alg HS256, typ JWT), payload = `UserSessionPayload: { id, email, role, permissions: string[], districtId: string|null, iat, exp }` — no password or refresh-token data ever goes into the JWT. Signature is HMAC-SHA256 using `JWT_SECRET` (separate from `JWT_REFRESH_SECRET` used for refresh tokens, so compromising one secret doesn't compromise the other token type).